Manipulación de Cadenas y Punteros: Control de Acceso en la Memoria RAM

Estudiante: Ian Quiroga **Curso:** 5.º año — 3.ª División (A-B) **Materia:** Laboratorio de Programación (LPR) **Institución:** E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López **Docente:** Mansilla Muñoz, York Elías **Actividad:** 4 — Manipulación de cadenas y punteros en C/C++ **Fecha:** 2026

Introducción

Para comprender cómo funciona la programación de bajo nivel es necesario analizar qué ocurre en la memoria RAM de la computadora. La RAM se organiza como **casilleros numerados** con direcciones físicas únicas expresadas en formato hexadecimal, y cada variable del sistema ocupa exactamente uno de esos casilleros.

El objetivo de esta actividad fue desarrollar un **sistema básico de control de acceso por edad** en C++, manipulando los datos en la memoria RAM mediante el uso directo de punteros y desreferenciación, y observando en pantalla las direcciones reales donde quedaron alojados los datos.

El sistema pide la edad al usuario, la guarda directamente en el casillero de memoria de una variable y decide si el acceso se aprueba o se rechaza según si la persona es mayor o menor de 18 años.

Desarrollo

El modelo mental: casilleros y direcciones

Concepto Significado
Memoria RAM Espacio físico de almacenamiento temporal donde residen las variables durante la ejecución del programa
Dirección hexadecimal Representación numérica en base 16 (que inicia con `0x`) para identificar un casillero exacto de la RAM
Puntero (pointer) Variable que guarda la dirección de memoria de otra variable del sistema
Operador de dirección (&) Devuelve la dirección física en la RAM donde está guardada una variable
Desreferenciar (*) Operación para leer o modificar el valor contenido en la dirección de memoria apuntada

El ejemplo central de la actividad son dos líneas:

```
int edad = 0;
int* p = &edad;
```

`edad` es una variable común que ocupa un casillero. `p` es un puntero que **no guarda la edad**: guarda la dirección hexadecimal de ese casillero. Si `edad` se aloja, por ejemplo, en la dirección `0x61fe1c`, entonces `p` vale `0x61fe1c`.

Diferencia entre `&` y `*`

El operador **ampersand** (`&edad`) obtiene la *ubicación física* en la RAM, mientras que el **asterisco** (`*p`) accede al *contenido* de esa dirección. Son operadores complementarios: uno lleva hacia la direccion, el otro vuelve desde la direccion hacia el valor.

La diferencia se nota directamente en la salida por pantalla:

Instrucción Qué imprime
`cout << p;` La **dirección** hexadecimal del casillero (ej: `0x61fe1c`)
`cout << *p;` El **valor** almacenado en ese casillero (ej: `21`)

Si se mostrara solo `p`, la consola imprimiría la dirección hexadecimal, que es precisamente el dato que la actividad pide evidenciar.

El error crítico: escribir sin desreferenciar

```
cin >> *p;   // Correcto
cin >> p;    // ERROR GRAVE de compilación
```

Es **obligatorio** desreferenciar el puntero al usar `cin` para modificar el valor de la variable original. Sin el asterisco, la instrucción no intentaría guardar la edad en el casillero de `edad`: intentaría **sobrescribir la dirección física de memoria que el propio puntero guarda**. El resultado sería que `p` deje de apuntar a `edad` y apunte a una dirección arbitraria leída del teclado. Si llegara a compilar, cada acceso posterior a `*p` leería y escribiría en un casillero ajeno, corrompiendo datos del programa.

Usando `cin >> *p;`, el asterisco le indica al compilador que debe guardar el número ingresado **directamente en el casillero de memoria original de** `edad`.

Por eso el orden de las dos líneas es fundamental: `int* p = &edad;` debe ejecutarse **antes** del `cin`. Si se invirtiera, el puntero guardaría basura.

El condicional con puntero desreferenciado

```
if (*p >= 18)
```

La condición evalúa el **valor** alojado en la memoria, no la dirección. El asterisco es indispensable: `if (p >= 18)` compararía la dirección hexadecimal contra el número 18 y devolvería un resultado sin sentido.

Según el resultado, el programa informa el acceso y muestra siempre la edad registrada y su dirección física, para poder verificar en la captura que el dato efectivamente quedó en la RAM y no en una copia.

La representación hexadecimal

Las direcciones de memoria se muestran en base 16, antecedidas por `0x`. El sistema operativo utiliza notación hexadecimal en lugar de decimal por una razón que tiene que ver con la arquitectura del hardware: las direcciones en las computadoras modernas maneja bloques de 8 bits (un byte). Como $8 = 2^3$, la base 16 (un dígito hexadecimal representa exactamente 4 bits) hace que **un solo carácter hexadecimal describa medio byte, y dos caracteres describan un**

byte completo. En otras palabras, el hexadecimal es la base que encaja exactamente con el tamaño de las direcciones reales, lo que reduce a la mitad los caracteres necesarios frente a la base decimal y evita conversiones confusas durante la depuración.

Conclusión

La actividad convirtió el concepto abstracto de puntero en algo visible: la dirección `0x61fe1c` que aparece en pantalla es la prueba física de que el dato realmente ocupa un casillero de la RAM.

Las conclusiones principales son tres. Primero, un puntero no contiene el dato: contiene **dónde está el dato**, y esa distinción es la base de toda la programación de bajo nivel. Segundo, desreferenciar no es opcional cuando se quiere modificar la variable original: omitir el `*` no produce un error visible sino corrupción silenciosa de la memoria. Tercero, la notación hexadecimal de las direcciones responde a la arquitectura del hardware, y no es una simple convencion estetica.

Entender estos tres puntos es lo que permite después trabajar con estructuras, vectores y archivos sin sentir que se esta tocando algo abstracto: en todos los casos se sigue tratando de leer y escribir en casilleros concretos de la RAM.

Referencias

• cppreference.com. (2024). *Pointer*. C++ reference.
<https://en.cppreference.com/w/cpp/language/pointer>

• cppreference.com. (2024). *Address-of operator* `&`. C++ reference.
<https://en.cppreference.com/w/cpp/language/operator_address_of>

• Microsoft. (2024). *Pointers (C++)*. Microsoft Learn.
<https://learn.microsoft.com/es-es/cpp/cpp/pointers-cpp>

• Tanenbaum, A. S., & Austin, T. (2013). *Structured computer organization*
(11.ª ed.). Pearson.

• GeeksforGeeks. (2024). *Pointers in C++*. <https://www.geeksforgeeks.org/pointers-in-cpp/>